

Architecture Overview

Introduction

The OptArrow system is a sophisticated architecture designed to handle optimization problems, specifically Linear Programming (LP) and Quadratic Programming (QP), through a combination of Python and Julia engines. This architecture is structured to facilitate efficient data processing and computation by leveraging modern data serialization formats and communication protocols.

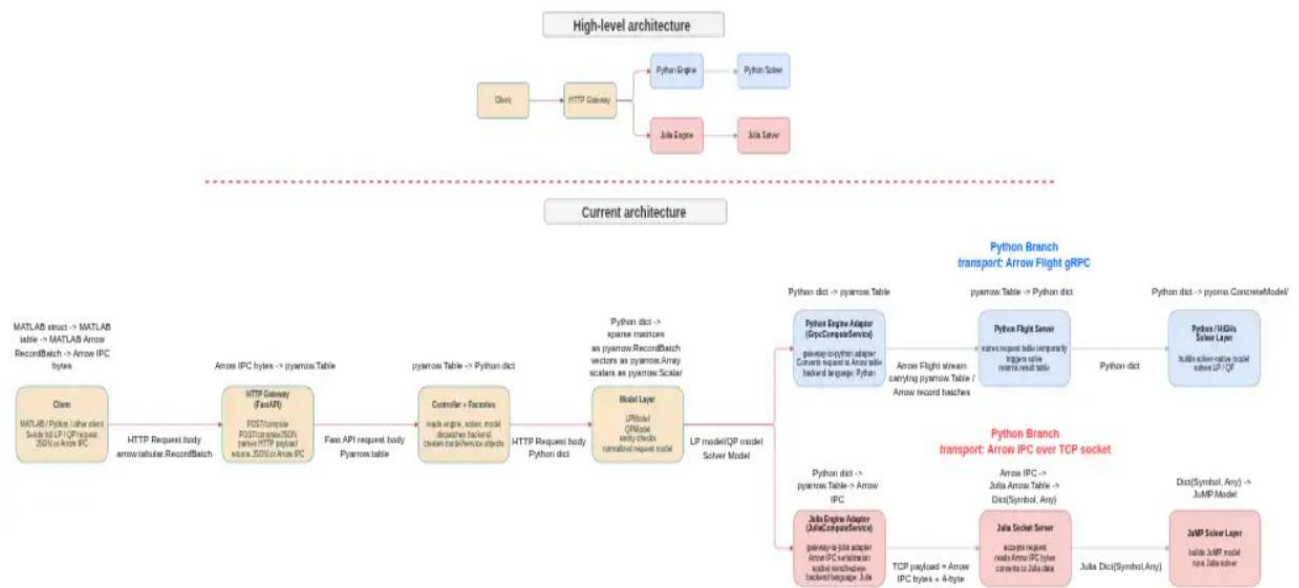


Figure 1: High Level Architecture

System Elements and Components

At the heart of the OptArrow architecture is the HTTP Gateway, implemented using FastAPI, which serves as the entry point for client requests. Clients, which can be MATLAB, Python, or other systems, send optimization requests in JSON or Arrow IPC format. The gateway processes these requests, parsing the HTTP payload and returning results in the same formats.

The architecture is divided into two main computational engines: the Python Engine and the Julia Engine. Each engine has its own adaptor, namely the GrpcComputeService for Python and the JuliaComputeService for Julia, which handle the conversion of requests into a format suitable for their respective backend languages. The Python Engine utilizes Arrow Flight gRPC for communication, while the Julia Engine employs Arrow IPC over TCP sockets.

Within the Python Engine, the Python Flight Server temporarily stores request tables and triggers the solving process using the Python/HiGHs Solver Layer. This layer builds solver-native models and solves the optimization problems. Similarly, the Julia Engine uses the JuMP Solver Layer to build JuMP models and execute the Julia solver.

Interfaces and Network Layout

The external interface of the system is the HTTP Gateway, which receives POST requests from clients. Internally, the system interfaces are defined by the communication between the gateway and the computational engines. The Python Engine communicates via Arrow Flight gRPC, a high-performance RPC framework, while the Julia Engine uses Arrow IPC over TCP, which involves serialization and deserialization of data into Arrow IPC bytes.

The architecture diagram illustrates these interfaces, showing how data flows from the client through the gateway to the respective engine adaptors and finally to the solver layers. The transport layers are explicitly defined, ensuring clarity in the network layout and data handling processes.

Allocation of Functional Requirements

The functional requirements of the OptArrow system are allocated across its components to ensure efficient processing and computation. The HTTP Gateway is responsible for receiving and parsing client requests, while the Controller and Factories within the system read the engine, solver, and model information to dispatch the appropriate backend services.

The Model Layer, which includes LPModel and QPModel, performs sanity checks and normalizes the request models. This ensures that the data is in the correct format before being processed by the computational engines. The Python and Julia Engines are tasked with converting requests into Arrow tables and executing the solvers, thereby fulfilling the core computational requirements of the system.

Data Flow and Dynamic Behaviors

The data flow within the OptArrow system is a journey of transformation and computation. Initially, client requests are received as HTTP request bodies, which can be either JSON or Arrow RecordBatch. These requests are parsed into Python dictionaries and then converted into PyArrow Tables.

Within the Python Engine, the data is further transformed into sparse matrices, vectors, and scalars as PyArrow RecordBatch, Array, and Scalar, respectively. The Python dict is then converted into a Pyomo ConcreteModel, which is used by the solver to build and solve the optimization problem.

In the Julia Engine, the Arrow IPC bytes are converted into Julia Arrow Tables and then into Julia dictionaries. These dictionaries are used to build JuMP models, which are subsequently solved by the Julia solver. The results are then serialized back into Arrow IPC format and returned to the client.

This dynamic behavior of data transformation and computation is crucial for the system's ability to handle complex optimization problems efficiently.